Integrating Embedded Sensors D25 ISP with WPI FSAE MQP

Philip Watson

Introduction:

This year, the WPI FSAE team wanted to improve the existing car from last year and start preparing for the following years, when they hope to do a driverless car. However, in order to do this, the various systems of the car need to be accurately modeled using data we collect about the car. One of the most crucial systems that needs to be modeled is the tires. This can be done using tire testing data, wheel speed, and the steering angle of the car. The wheel speed is also important for the front wheels, specifically because they are free spinning and the RPM cannot be approximated like the rear wheels can be through the motor and inverter. This data is also important for not only driverless, but also traction control. This is why one of the sensors I took on to get working for my ISP was the wheel speed data using the onboard ADC on the wheel speed sensors, which were designed by Silas Gagnon, the I/O Board code for them, and the mounts they needed. I also worked on getting the steering angle sensor, which was designed by Harrison Brown, functional for this goal as well.

The WPI FSAE team also wants to use sensors to verify the simulations that have been ran on specific components of the car. One of these systems we wanted to verify the simulations of was the aerodynamic surfaces that were designed for the car, using an array of air pressure sensors on the top and bottom of the surfaces. For these, I designed the PCBs and created the code for them to run.

Steering Angle Sensor:

How it works:

The steering angle sensor uses the MA732GQ-Z, which has 360 hall effect sensors in a ring that is able to output an angle of a diametrically magnetized cylindrical magnet that is placed over it coaxially. The magnet is attached to the pinion gear of the steering rack via a rectangular tab inside the pinion gear and the sensor is attached to the outside of the rack, from this the steering angle can be measured.

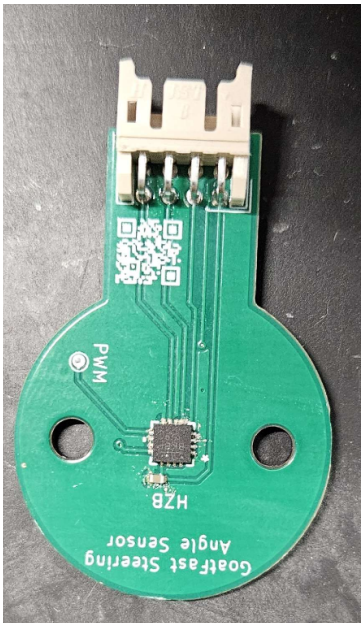
Work done:

In D-Term, I assembled the PCB for the steering angle sensor and worked on getting it to work with SPI to communicate with the I/O Boards connected to the

telemetry loop. However, the PCBs were repurposed to be used in the pedals due to the sensors that were in them falling out of sync with each other and causing the car to fail. After they were repurposed there were no extra hall effect sensors, because of this I was unable to finish working on the steering angle sensor.

Work to do:

The current mount for the steering angle sensor that was designed does not fit in the pinion gear, the mount also comes out far enough from the pinion gear that the frame intersects it, and would require new longer bolts to be sourced for the steering rack. When redesigning the mount it should be thinner, this can be achieved by using a smaller magnet which can fit inside the pinion mount and geometry that would require less thickness. To prevent the team from having to take apart the steering and source new bolts for them, the team should use the 8-32 mounting holes that Kaz Technologies uses to mount their own steering sensor that can be bought from them.



Steering Angle Sensor PCB



Kaz Technologies Steering Rack with Red Circles Marking the Steering Angle Sensor Mounting Holes

Wheel Speed Sensor:

How it works:

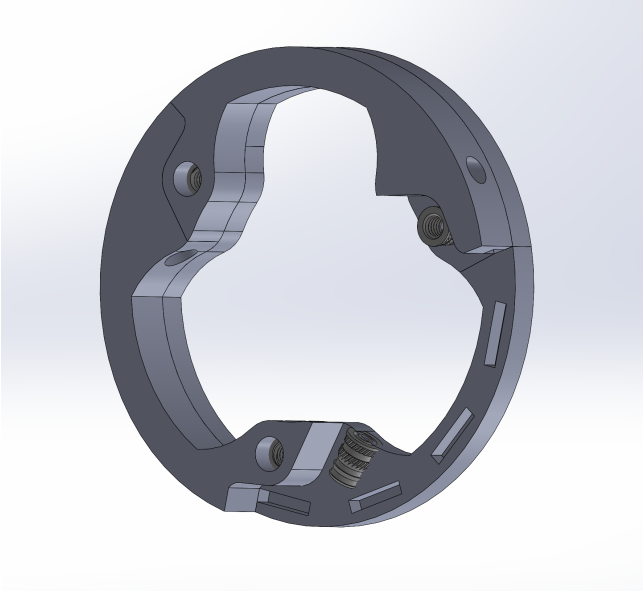
The wheel speed sensors have two hall effect sensors spaced a set distance away from each other and feed into an ADC that connects to an I/O Board over SPI. There is also a ring of 12 alternating magnets that connects to the differential coupler of the rear wheels. The hall effect sensors then measure how quickly the magnetic field fluctuates and thus the speed of the wheels.

Work done:

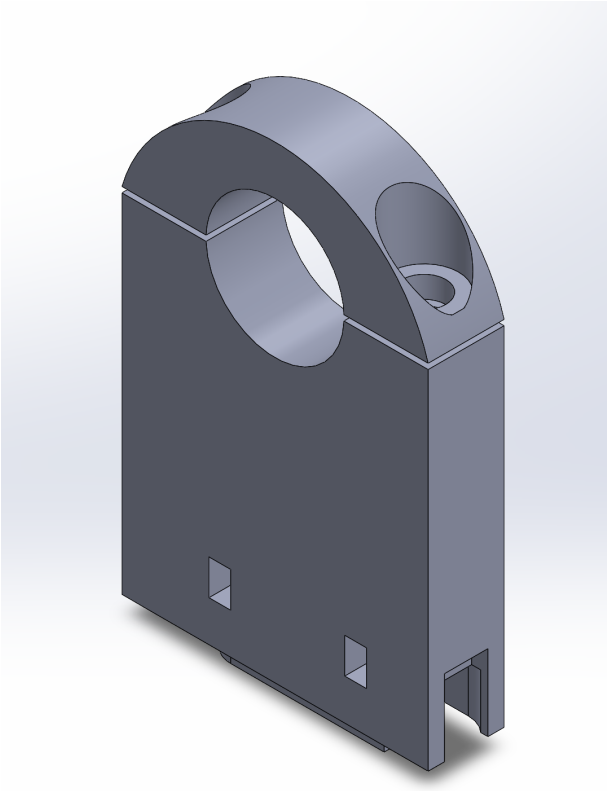
During D-Term, I built two additional wheel speed boards because the already built boards had damaged ADCs on them, and we were unable to use the test points that go directly to the hall effect sensors on the board and connect their analog signals to the ADC in the STM32 on the I/O Boards due to other peripherals needing them. Once the boards were built I was able to program asynchronous control code that would set up the ADC, take and log values, send wheel speed values over the CAN bus, and allow additional sensors to run on the same I/O Board.

I also created the mounts for the wheel speed sensor boards and the magnet rings. The wheel speed sensor boards I designed have adjustable height because they need to operate within a +/- 2 mm range in height away from the magnet ring. However, the absolute distance between the part of the frame the sensor mounts to and where the magnet ring is changes due to the turnbuckles that position the motor. To have adjustable height, I designed the mount to use 1 mm and 3 mm shims underneath the board to adjust the spacing. I also designed the magnet ring to fit around the differential coupler, I chose this design because it has a lobed shape which allows for interfacing geometry

preventing the magnet ring from slipping. To prevent the magnet rings from slipping along the differential coupler, I put grub screws on the ring that interface with it.



Magnet Ring Model in SolidWorks



Wheel Speed Sensor Holder

Work to do:

On the software side, the code for the math to convert hall effect sensor data to wheel speed still needs to be transferred from Silas' initial code that read the values of the hall effect sensors with the I/O Board ADC.

On the mechanical side, in the future there should be mounts on the uprights for a sensor and the magnets on the inside of the wheels or on the axle of the wheel. This is because they could be better integrated on the front wheels and rear wheels in the same manner. They would also not be subject to how screwed in the turnbuckles are, and thus would not need to have height adjustability .

Aerodynamic Sensor:

How it works:

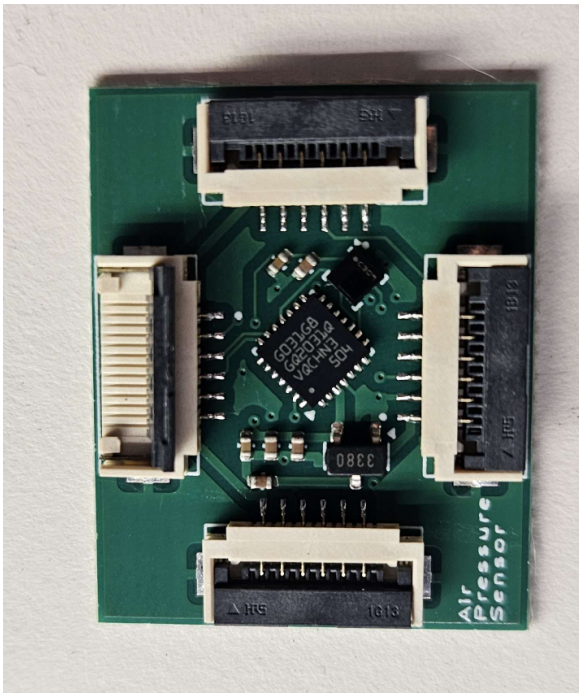
The aerodynamic sensors are an array of sensors on the top and bottom of aerodynamic surfaces that measure the air pressure change. These sensors are connected to each other in a UART daisy chain and then that then connects to an I/O Board that sends the data on the telemetry loop CAN bus.

Work done:

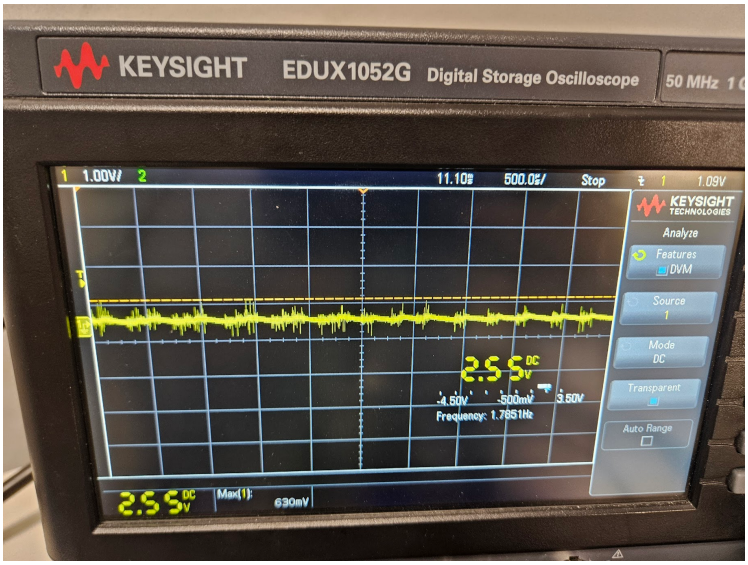
This term, I designed the sensor PCB and had them manufactured through JLC PCBA. One of the important aspects of these sensors is that they should not significantly affect the airflow of the aerodynamic surfaces. To do this, I use flat flex connectors when connecting to the boards and made the sensor using 0.8 mm FR4. I also choose to use an STM32G031G8U6 on board the sensors to interface with the LPS22HBTR air pressure sensor over SPI and enable a UART daisy chain. Another aspect is that the sensors should be able to work with different shaped arrays, so I added two UART Tx ports at right angles from each other to allow for any sized rectangular arrays. I also created adapter boards to enable them to be programmed and debugged with a flat flex cable and to adapt two flat flex cables to the standard 8 pin JST connectors that are used on the I/O Boards.

I initially tried to communicate with the LPS22HBTR over I2C, however I didn't add pull up resistors on the board and the internal pull up resistors on the STM32 had too much resistance, so I switched to using SPI because SPI is push-pull, which does not require external pull-ups. Furthermore, I chose to set up the LPS22HBTR registers to take an initial measurement upon boot and then only give the pressure offset from that because relative pressure change is what needs to be measured overtime and to each other because that will determine the effectiveness of the aerodynamic surfaces. I have written the code to interface with the LPS22HBTR and to create a UART daisy chain,

however I have been unable to debug the code and see what the data output of the LPS22HBTR is like.



Aerodynamic Sensor PCB



I2C Clock signal only using the STM32 internal pull-ups

Work to do:

The code on the air pressure sensors needs to be debugged and data needs to be collected to see how the data can be compressed, so that it takes up minimal space on the CAN bus. Assuming that the data from each sensor on each side of an aerodynamic surface does not vary much and thus instead of keeping the entire 24 bit data points, you can just keep 8 or 16 bits after the initial data point. The code also needs to be written for taking the data from the UART daisy chain and sending it onto the CAN bus.

Reflection:

Overall, I learned a tremendous amount about programming embedded systems and also how to debug them myself using tools such as an oscilloscope and GDB. I also learned more about PCB design, especially when putting microcontrollers and sensors on boards. My ability to read a datasheet has also increased dramatically, from being too intimidated to approach them to understanding how to read one efficiently. Having now done a project in embedded systems has shown to me how interesting it can be to work with them and makes me want to continue to learn more and do more projects with them in the future.

Acknowledgements:

Thank you to Professor Michalson for allowing me to do this ISP with the WPI FSAE MQP Team and thank you to the rest of the team for helping me out and answering all my questions, especially Eric, Silas, Kate, and Carson.